



Aqsa Khalid
Lecturer
School of Computing

CS3002 Information Security



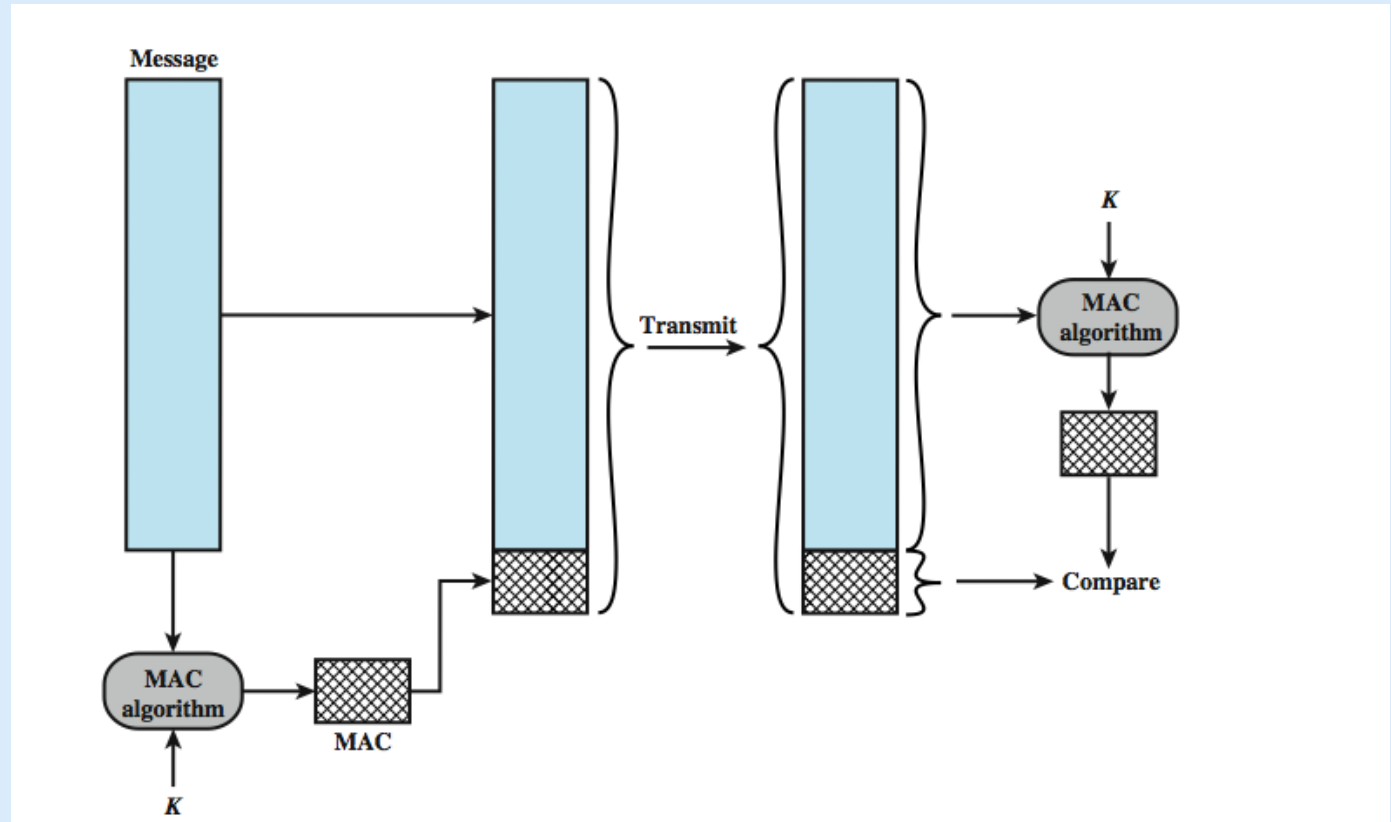
Message Integrity & Authenticity Protection

Message Authentication



- **Goal:** Protection against active attacks (alteration, falsification)
- Receiver verifies received message is authentic
 - a) contents are unaltered (integrity is protected)
 - b) comes from an authentic source
- **Message Authentication Code (MAC):** a small block of data appended to the message, used for authenticity checking at receiver
 - Note the difference from checksum which only protects against accidental (non malicious) data errors

Message Authentication Code



MAC algorithm will always have a **secret** (K) as input. Checksums do not require any input other than data.



MAC Algorithms

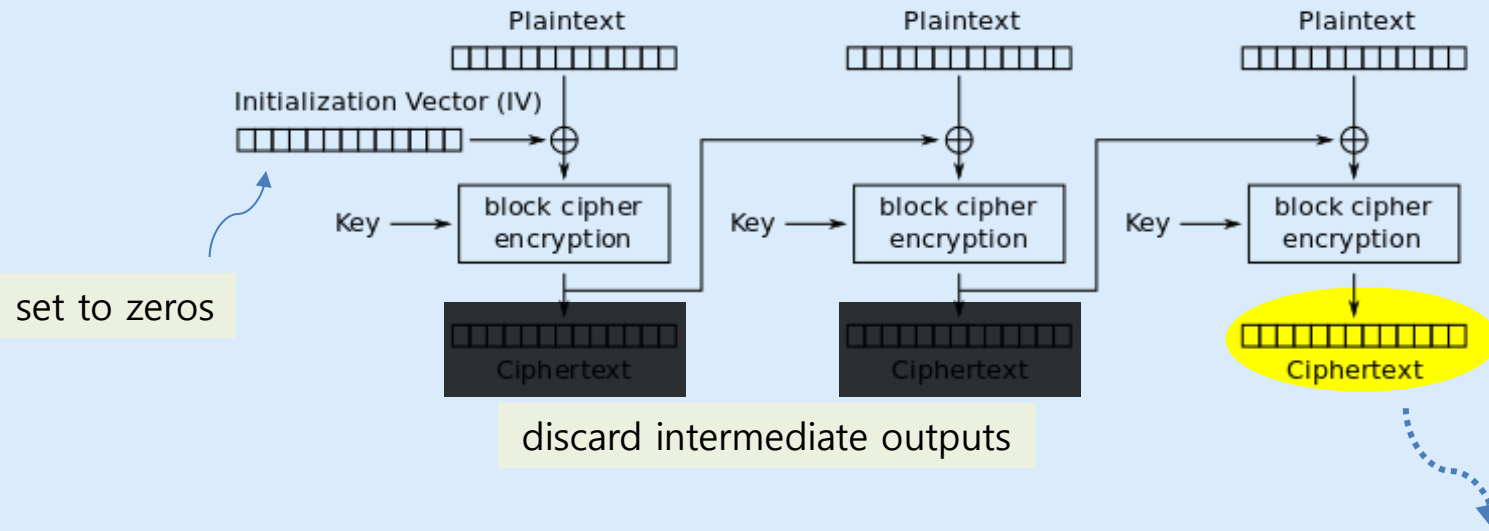


- There are mainly two types of MAC algorithms
- 1) Block cipher based MACs**
 - Use a block cipher (like AES) to generate the message authentication code.
 - CBC-MAC is one of the many options
 - 2) Hash function based MACs**
 - Use cryptographic hash functions combined with a secret key

CBC MAC



- In this algorithm, any symmetric block cipher is used in the Cipher-Block-Chaining mode
 - only sender & receiver have the key needed

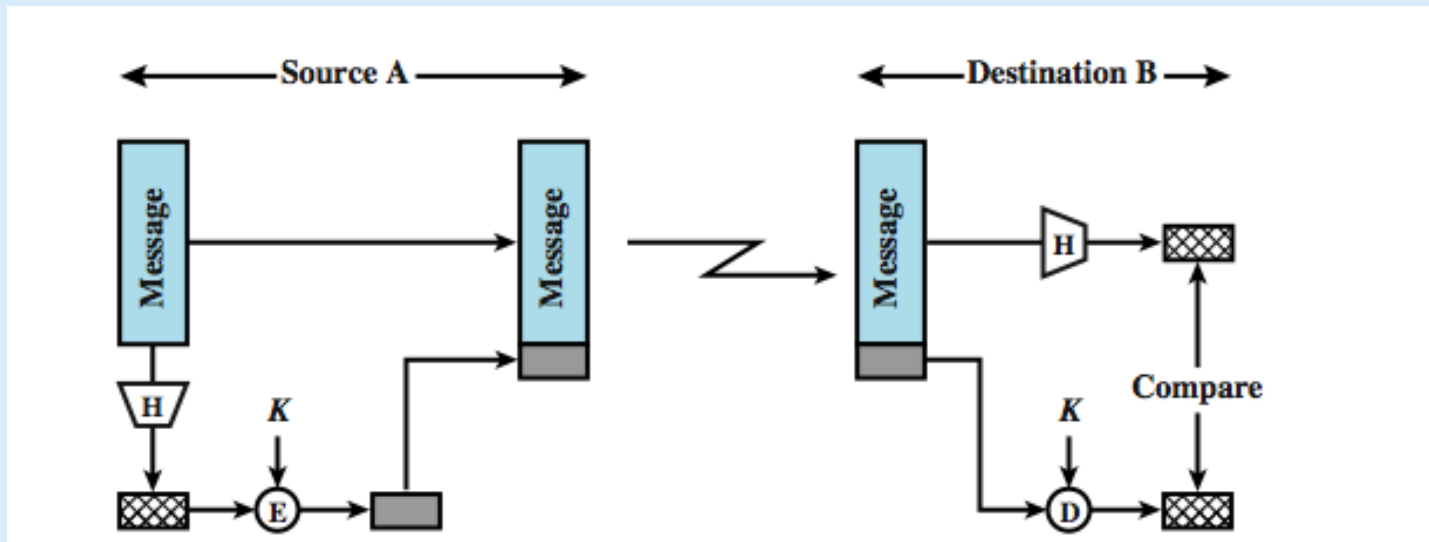


Final block output is dependent on all input blocks (i.e. the whole message), so it can act as a MAC

Hash-then-encrypt MAC



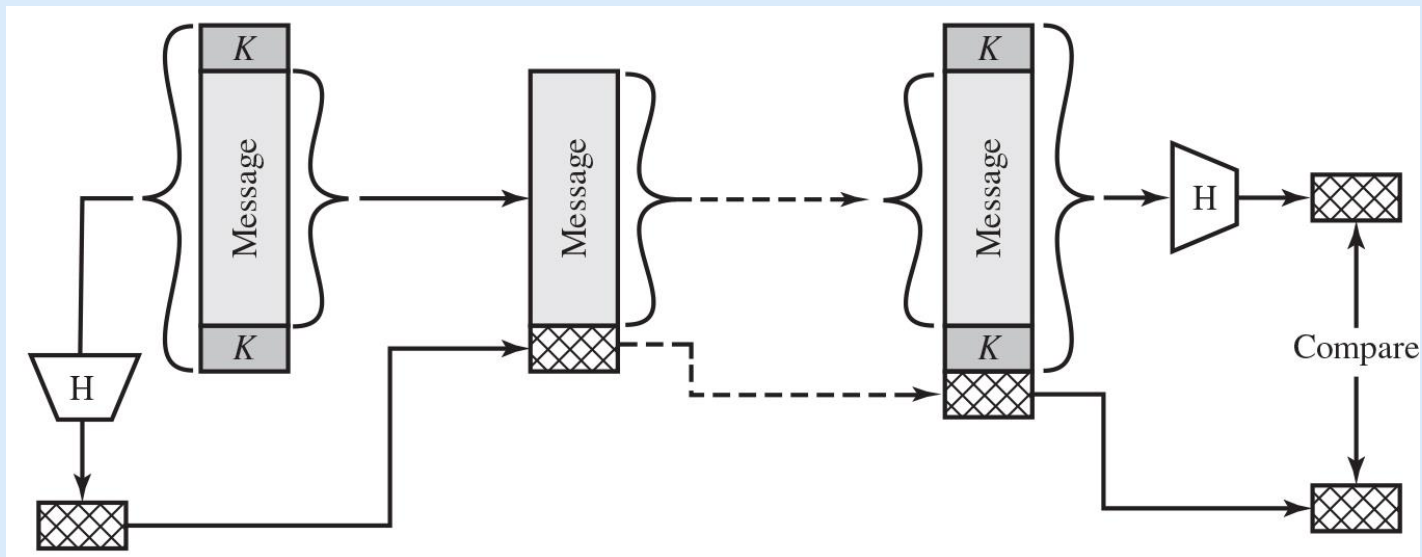
- In these category of algorithms, we may combine encryption with hash functions
 - e.g. first compute the hash of data and then encrypt the result with secret key



Keyed-Hash MAC



- can also skip encryption altogether and just use hash functions
 - Bracket the data with secret key before hashing



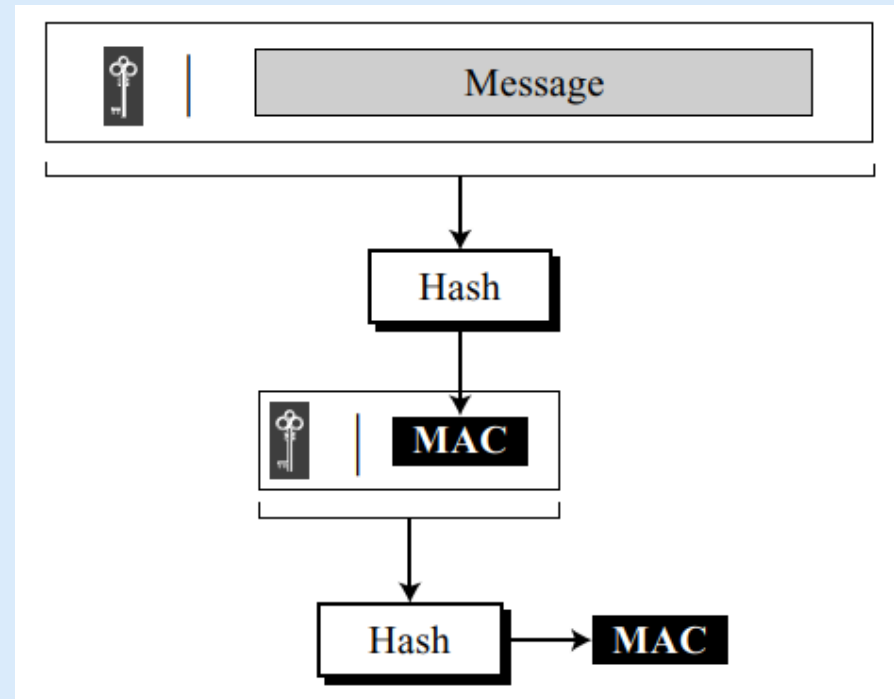
Nested hashing MAC



- Hash-based MACs depend on cryptographic security of hash function (discussed later). Some hash functions are prone to so called *length-extension attacks*. To avoid that loophole, we can **apply hashing twice**.

Firstly, key is prepended to message and hashed to get an intermediate MAC.

Then the process is repeated once more to get final MAC.



HMAC: A standard nested hashing MAC



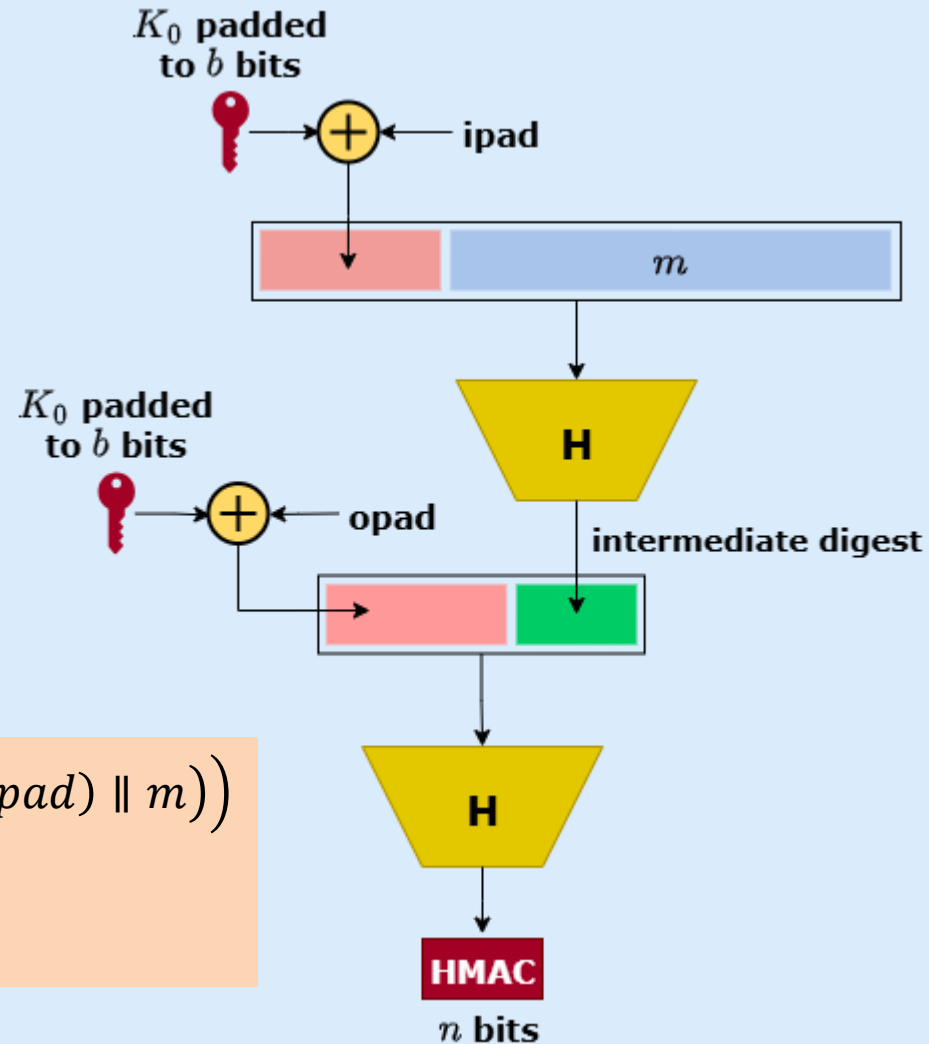
NIST has standardized a variant of nested hashing MAC algorithm.

- It needs a message m and a secret key K as inputs.
 1. Pick a cryptographic hash function H with block size b bits (e.g. SHA-256 which has 512 bit internal block size).
 2. If the secret key K is longer than b , compute $K_0 = H(K)$, otherwise set $K_0 = K$.
 3. Right-pad (extend) K_0 with zeros to make a b -bit block.
 4. XOR the padded K_0 with a constant called **ipad** (inner pad). The value of ipad is bit sequence 00110110 (0x36) repeated $b/8$ times.
 5. Prepend the resulting b -bit block to the message.
 6. Hash the result to create an intermediate digest.

HMAC: A standard nested hashing MAC



7. Redo step 4 by a different constant **opad** (outer pad), which is bit sequence 01011100 (0x5C) repeated $b/8$ times.
8. Prepend this block to the intermediate digest of step 6.
9. Hash the result of step 8 with the same hashing algorithm to create the final n -bit HMAC.



$$\text{HMAC}(m, K) = H\left((K_0 \oplus \text{opad}) \parallel H((K_0 \oplus \text{ipad}) \parallel m)\right)$$

$$\text{where } K_0 = \begin{cases} H(K) & \text{if } \text{len}(K) > b \\ K & \text{otherwise} \end{cases}$$

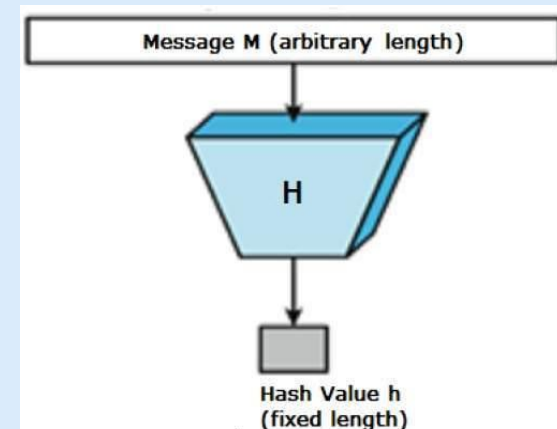
Cryptographic Hash Functions



Hash Functions



- These are one-way transformations
- Produce 'message digests' or 'fingerprint' of the input data
- Input to hash functions can be variable size, usually large number of bits.
- Output is small, fixed number of bits
- **Collisions:** Two different messages having the same hash
 - Since hashing is a many-to-one function, collisions are unavoidable



Cryptographic Hash Functions



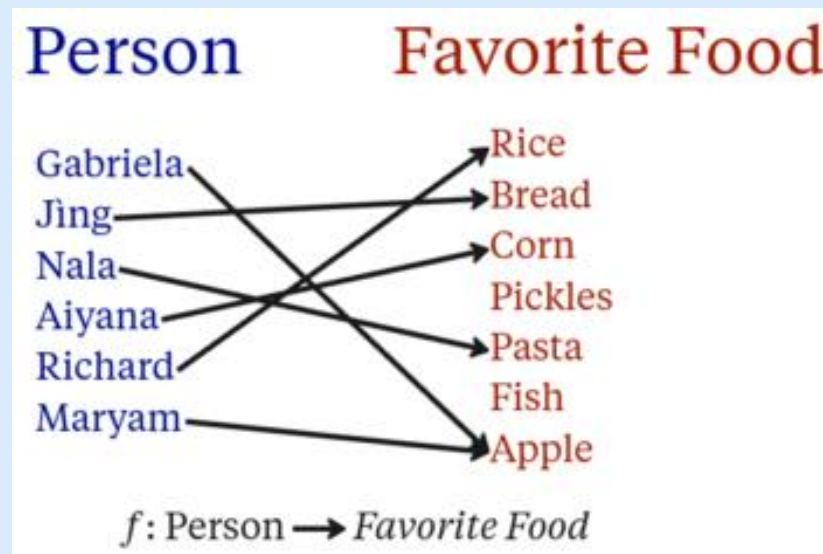
Not all hash functions can be used in security algorithms. So called **cryptographic hash functions** need to have the following properties

- 1) Output is a (pseudo-)random combination of 1's and 0's distributed variably with a proportion of 50% each
 - A single bit change in input must change the output by roughly 50%
 - When flipping two bits in the input, the bits flipped in the output will be totally unrelated to which bits would flip if you just changed the input bits one by one.

Aside: Images and Pre-images



Consider the following many-to-one function



Maryam's **image** is Apple

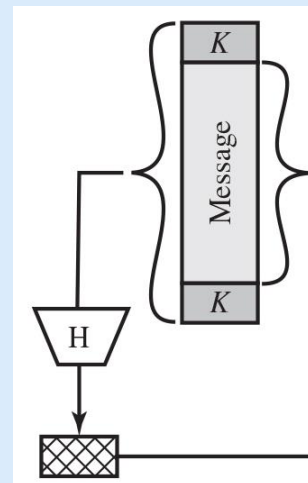
Preimage of Apple is {Gabriella, Maryam}

Cryptographic Hash Functions



2) Pre-image Resistance (one-way property)

- Given a hash function h and a digest y , it must be extremely difficult for Eve (attacker) to find any message, M' , such that $y = h(M')$.
- In other words, it should be virtually impossible to reverse the hashing (find preimage of a digest).
- Otherwise, an attacker could discover a secret value K , that was used to generate a MAC.



Cryptographic Hash Functions



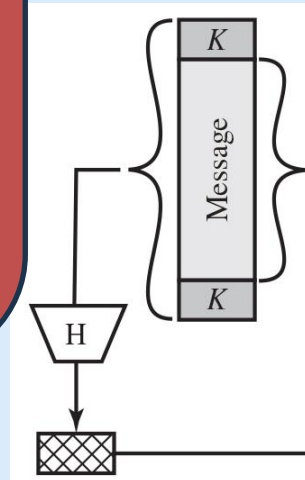
2) Pre-image Resistance (one-way property)

- Given a hash function h and a digest y , it must be extremely difficult for Eve (attacker) to find any message M' such that $y = h(M')$.

- In order to do
- reverse
- Other
- K, t

Analogy

In a class of students, find one who has a specific birthday (say 15th Sep)

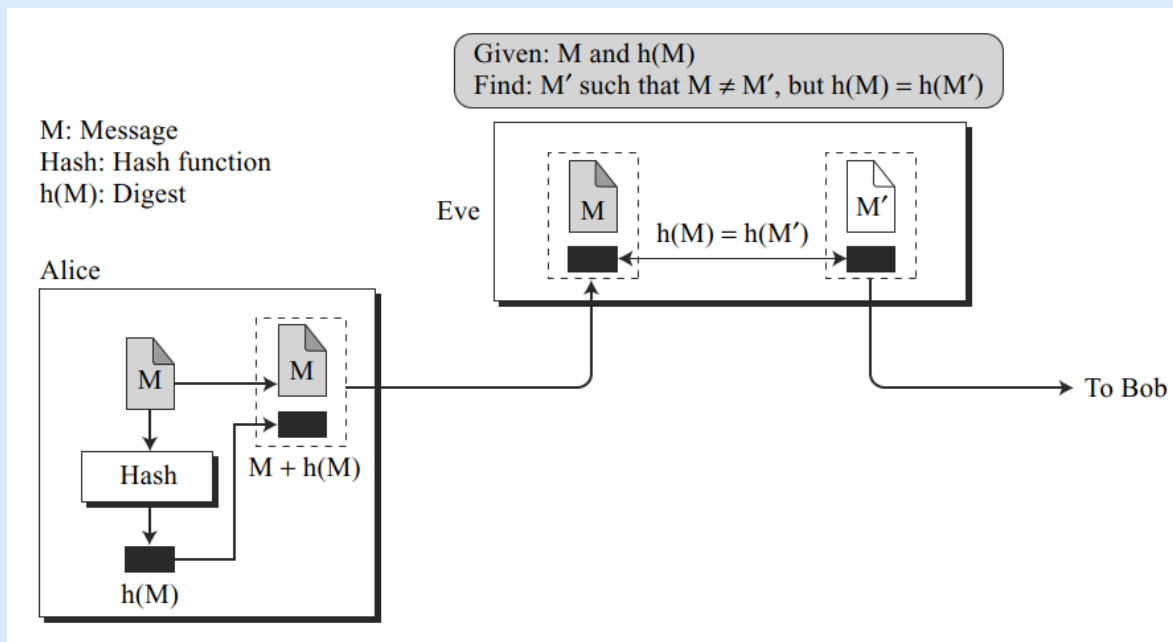


Cryptographic Hash Functions



3) Second Preimage Resistance

- Given a specific message and its digest, it must be extremely difficult to create another message with the same digest.



Without 2nd preimage resistance, Eve can silently replace a message with a forged one.

Cryptographic Hash Functions



3) Second Preimage Resistance

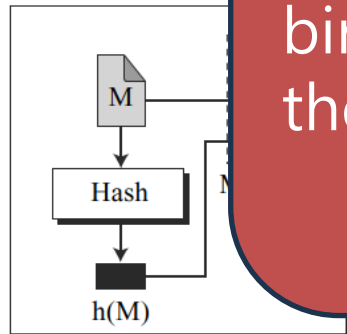
- Given a specific message and its digest, it must be extremely difficult to create another message with the same digest.

Analogy

In a class, there is a student Ali, with birthday 20th Sep. Find another one from the class, with the same birthday.

M: Message
Hash: Hash function
 $h(M)$: Digest

Alice



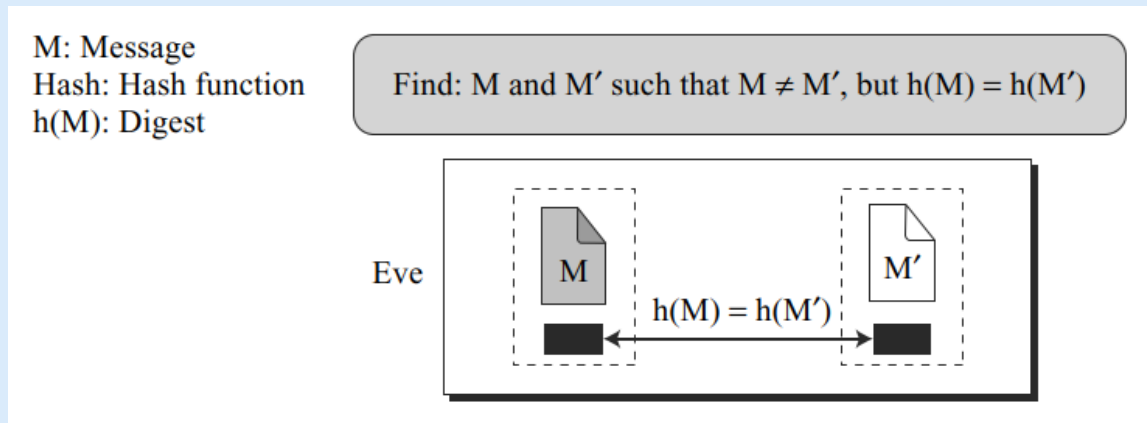
Second Preimage Resistance
Given a message, an attacker can silently find another message with a same digest.

Cryptographic Hash Functions



4) Collision Resistance

- Eve cannot find two messages (from scratch) that hash to the same digest.



- This type of attack is much easier to launch than the previous kind (forgery). Why?

Cryptographic Hash Functions



4) Collision Resistance

- Eve cannot find two messages (from scratch) that hash to the same digest.

Analogy

In a class, find ANY two students with the same birthday.

- This is the birthday paradox, which states that in a group of 23 people, there is a 50% chance that two people share the same birthday.

Cryptographic Hash Functions



4) Collision Resistance

- Eve cannot find two messages (from scratch) that hash to the same digest.

Analogy

In a class, find ANY two students with the same birthday.

For a class of only 23 students, probability of finding two students with same birthday is 50%. When class size increases to 70, you are almost sure (99.9%) to find two such students!

[This is called birthday paradox]

- This
prev

the

Cryptographic Hash Functions



- Collision resistance becomes relevant when one party generates a message for another party to sign. e.g.
 1. Alice asks Bob to prepare a cheque in his name, and she would sign it.
 2. Bob finds two messages (cheques) with the same hash — one of which requires Alice to pay a small amount and one that requires a large payment.
 3. Alice signs the first message (appends a MAC), and Bob is then able to claim that the second message is authentic.

Attacks against Hash functions



- Cryptanalysis: exploit logical weakness in algorithm to reverse the hashing
- Brute force: trial many inputs to find preimage of a given digest.
 - Strength is proportional to size of digest. For a hash function with n bit output, brute force strength is proportional to:

Preimage & 2 nd preimage resistance	2^n
Collision resistance	$2^{n/2}$ (or $\sqrt{2^n}$)

Commonly used hash functions



- MD5: older, 128-bit hash.
 - Proposed in 1992
 - Now considered insecure because it was proved to be not collision-resistant (but preimage attacks are still impractical)
- SHA-1: very widely used, 160-bit hash.
 - Now also considered vulnerable due to mathematical weaknesses
- SHA-2: more recent, bigger size, more secure
 - multiple variations: SHA-256, SHA-384, SHA-512
- SHA-3: most recent

Digital Signatures



Digital Signatures

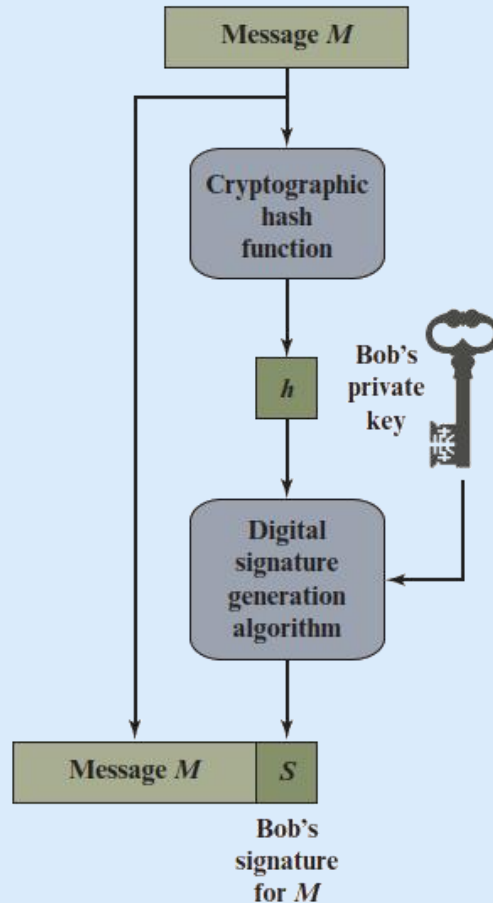


- Combines a hash with a PKC algorithm
- To sign
 - hash the data
 - encrypt the hash with the sender's private key
 - send data, signer's name and signature
- To verify
 - hash the data
 - decrypt the signature with the sender's public key...
 - the result of which should match the hash

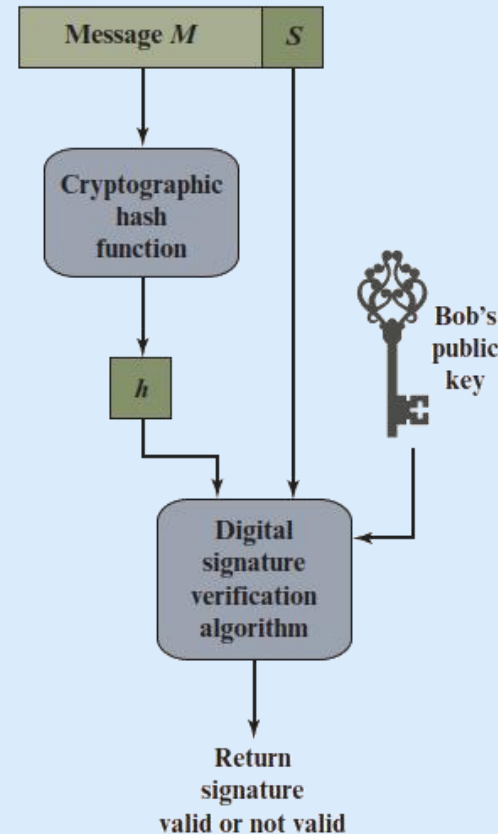
Digital Signatures



Bob
signs a message



Alice
verifies the signature



Digital Signature Schemes



- Previous slide only shows a general flowchart of signature calculation.
- But there are lot of implementation details such as choice of hash function, choice of encryption algorithm, message padding etc.
- Many signature schemes have been proposed that include detailed calculation algorithms
 - e.g. RSA digital signatures, ElGamal scheme, Schnorr scheme, DSS etc.
 - Details of these are beyond course scope.

Digital Signatures vs MAC



- MAC proves
 - **message integrity** because any change in message will cause MAC verification to fail.
 - **message authenticity** because only the users having the secret key can compute the MAC.
- Digital signatures provide both, and additionally bring **non repudiation**: signer can not deny having signed the message, since it involves a private key.
 - However a trusted 3rd party will be required to maintain an archive of public keys of all users.

References



- Stallings CNS chap 11, 12
- Forouzan, Cryptography & Network Security – chap 11-12
- HMAC on Computerphile:
<https://youtu.be/wlSG3pEiQdc?t=254>